

Sequential RISC-V Processor Design

Team Name: OG

Team 37

Jai Srikar M (2024102041)

Mohammad Shabaj (2024102016)

Nelagonda Pavaneshwar (2024102063)

Contents

1	Introduction	2
2	Project Overview	3
2.1	Architecture Type	3
2.2	Supported Instructions	3
2.3	Top-Level Integration	3
3	Two-Part Development Model	4
3.1	Part-A: Control and Datapath Core	4
3.2	Part-B: Memory Subsystem	4
3.3	Interface Between Parts	5
4	Datapath Flow per Cycle	5
5	Architecture Blocks and Implementation Details	5
5.1	Program Counter (PC)	5
5.2	Instruction Memory	6
5.3	Register File	6
5.4	Control Unit	6
5.5	Immediate Generation	6
5.6	ALU Control	6
5.7	ALU Block	7
5.8	Data Memory	7
6	Branch and PC Update Logic	7
7	Testing and Verification	7
7.1	Testbench Behavior (seq_tb.v)	7
7.2	Simulation Commands	8
7.3	Progressive Verification Strategy	8
8	Key Design Decisions	10
9	Challenges Faced	10
10	Conclusion	11

1 Introduction

In this project, we designed and implemented a complete 64-bit single-cycle RISC-V processor conforming to a subset of the RV64I instruction set architecture. The objective was to translate theoretical datapath concepts from lectures into a structurally sound and functionally correct hardware implementation using Verilog.

Our design executes one complete instruction per clock cycle at the architectural level. This means that instruction fetch, decode, execution, memory access, and register writeback are all completed within a single clock period. While this simplifies control complexity, it requires careful combinational timing discipline and correct signal coordination.

To ensure modularity and manageability, we divided the implementation into two logical development phases:

- **Part-A:** Control logic and core datapath implementation.
- **Part-B:** Memory subsystem design and integration.

This separation allowed us to independently validate ALU operations and branch logic before introducing memory behavior, thereby significantly reducing integration complexity.

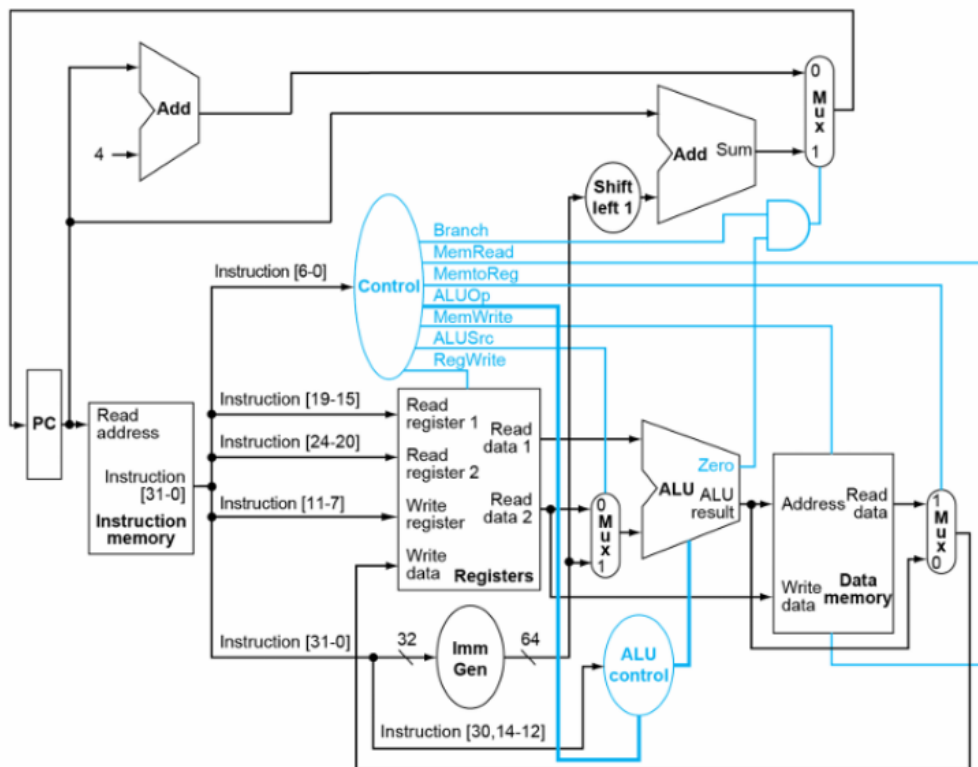


Figure 1: Overall architecture of the implemented single-cycle processor.

2 Project Overview

2.1 Architecture Type

The processor follows a classical single-cycle datapath architecture with the following characteristics:

- Single-cycle execution model.
- 64-bit datapath width (RV64I compliance).
- Byte-addressable memory.
- Big-endian byte ordering for both instruction and data memory.
- 32 general-purpose 64-bit registers.
- Architectural register `x0` permanently hardwired to zero.

The design deliberately avoids pipelining or multi-cycle optimizations in order to clearly reflect the textbook single-cycle architecture discussed in class.

2.2 Supported Instructions

We implemented the following subset of RV64I:

- **R-type:** `add`, `sub`, `and`, `or`
- **I-type:** `addi`
- **Load:** `ld`
- **Store:** `sd`
- **Branch:** `beq`

This subset was selected to ensure that arithmetic, logical, immediate handling, memory access, and control-flow mechanisms were all exercised.

2.3 Top-Level Integration

The top-level integration is performed in `processor.v`. This module instantiates and connects:

- Program Counter
- Instruction Memory
- Immediate Generator

- Control Unit
- ALU Control
- 64-bit ALU
- Data Memory
- Register File

All datapath routing, control signal wiring, and next-PC selection logic are implemented here.

3 Two-Part Development Model

3.1 Part-A: Control and Datapath Core

In Part-A, we implemented:

- PC update logic.
- Instruction fetch and field extraction.
- Opcode-based control decoding.
- ALU operation decoding.
- Immediate generation.
- Branch decision logic.
- Writeback path selection.

This phase ensured arithmetic and branch instructions functioned correctly before integrating memory.

3.2 Part-B: Memory Subsystem

Part-B focused on:

- 1024-byte data memory design.
- Big-endian 64-bit load/store mapping.
- Sequential write semantics.
- Combinational read semantics for single-cycle correctness.

3.3 Interface Between Parts

The integration signals were:

- To memory: `MemRead`, `MemWrite`, `address`, `write_data`
- From memory: `read_data`

4 Datapath Flow per Cycle

Each clock cycle performs:

1. PC provides instruction address.
2. Instruction memory outputs 32-bit instruction.
3. Fields (opcode, rs1, rs2, rd, funct3, funct7) extracted.
4. Control signals generated.
5. Immediate computed and sign-extended.
6. ALU operands selected.
7. ALU executes operation.
8. Memory accessed if required.
9. Writeback selected.
10. Register file updated.
11. PC updated.

5 Architecture Blocks and Implementation Details

5.1 Program Counter (PC)

The PC is a 64-bit register updated synchronously. It supports:

- Sequential execution via $PC+4$.
- Branch redirection via branch target.
- Reset initialization to zero.

5.2 Instruction Memory

Instruction memory:

- Byte-addressable.
- Initialized using `$readmemh`.
- Assembles 4 consecutive bytes into one 32-bit instruction.
- Uses big-endian ordering.

5.3 Register File

The register file:

- Contains 32 registers.
- Provides two combinational read ports.
- Provides one synchronous write port.
- Forces `x0` to remain zero.

5.4 Control Unit

The control unit decodes opcode and produces:

`RegWrite`, `MemRead`, `MemWrite`, `MemToReg`, `ALUSrc`, `Branch`, `ALUOp`

It defaults all signals to zero before case-based activation.

5.5 Immediate Generation

Immediate logic reconstructs:

- I-type immediates.
- S-type immediates.
- B-type branch offsets (with implicit LSB zero).

All are sign-extended to 64 bits.

5.6 ALU Control

ALU control interprets `ALUOp` and function fields to produce a 4-bit operation code.

5.7 ALU Block

The ALU is structurally composed of:

- 64-bit adder.
- Subtractor (two's complement).
- Bitwise logic blocks.
- Shift units.
- Comparison logic.

Outputs include flags: zero, carry, overflow.

5.8 Data Memory

The memory subsystem:

- Stores 1024 bytes.
- Writes 64-bit values as eight bytes.
- Reads combinationally.
- Ensures bounds checking.

6 Branch and PC Update Logic

Branch decision:

```
branch_taken = Branch & zero_flag
```

Next PC:

```
pc_next = branch_taken ? branch_target : pc_plus4
```

7 Testing and Verification

7.1 Testbench Behavior (seq_tb.v)

To validate functional correctness, we designed a structured testbench in `seq_tb.v`. The testbench performs the following tasks:

- Generates a periodic clock signal.
- Applies synchronous reset at the start of simulation.

- Executes the processor for a bounded number of cycles (`MAX_CYCLES = 300`).
- Dumps the final architectural register state into `register_file.txt`.

The output file contains:

- 32 lines representing registers `x0` through `x31`.
- One additional line indicating total cycle count.

This output format strictly follows the evaluation guidelines provided.

7.2 Simulation Commands

The project is structured so that the following commands are sufficient to build and run the processor:

- `iverilog seq_tb.v`
- `./a.out`

All Verilog modules are included via `'include` statements in `seq_tb.v`, ensuring compilation works exactly as specified in submission instructions.

7.3 Progressive Verification Strategy

We validated the processor in incremental stages:

1. **ALU Validation:** Verified `add`, `sub`, `and`, `or`, and `addi`.
2. **Memory Validation:** Tested `sd` and `ld` across multiple aligned addresses.
3. **Branch Validation:** Confirmed both branch-taken and branch-not-taken cases.
4. **Loop Execution:** Implemented backward-branch loop to verify negative immediate handling and PC redirection stability.

The following screenshots demonstrate our final testing workflow:

```
register_file.txt x
register_file.txt
1 0000000000000000
2 0000000000000000
3 0000000000000000
4 0000000000000000
5 0000000000000000
6 0000000000000000
7 0000000000000000
8 0000000000000000
9 0000000000000000
10 0000000000000000
11 0000000000000000
12 0000000000000000
13 0000000000000000
14 0000000000000000
15 0000000000000000
16 0000000000000000
17 0000000000000000
18 0000000000000000
19 0000000000000000
20 0000000000000000
21 0000000000000000
22 0000000000000000
23 0000000000000000
24 0000000000000000
25 0000000000000000
26 0000000000000000
27 0000000000000000
28 0000000000000000
29 0000000000000000
30 0000000000000000
31 0000000000000000
32 0000000000000000
33 |
```

Figure 2: Values stored in registers x0 - x31 before executing the program

```
Instructions.txt x
Instructions.txt
1 00
2 50
3 00
4 13
5 00
6 00
7 00
8 00
9 00
10 00
11 00
12 00
13 00
14 00
15 00
16 00
17 00
18 00
19 00
20 00
21 00
22 00
23 00
24 00
25 00
26 00
27 00
28 00
29 00
30 00
31 00
32 00
33 00
34 00
35 00
36 00
37 00
38 00
39 00
40 00
41 00
42 00
43 00
44 00
45 00
46 00
47 00
48 00
49 00
50 00
```

Figure 3: The program implemented

```
register_file.txt x
register_file.txt
1 0000000000000000
2 000000000000000f
3 ffffffff0000000b
4 000000000000000a
5 000000000000000a
6 000000000000000a
7 ffffffff0000000b
8 ffffffff0000000b
9 0000000000000000
10 0000000000000000
11 000000000000000f
12 ffffffff0000000b
13 0000000000000000
14 000000000000001e
15 0000000000000000
16 0000000000000000
17 0000000000000000
18 0000000000000000
19 0000000000000000
20 0000000000000000
21 0000000000000000
22 0000000000000000
23 0000000000000000
24 0000000000000000
25 0000000000000000
26 0000000000000000
27 0000000000000000
28 0000000000000000
29 0000000000000000
30 0000000000000000
31 0000000000000000
32 0000000000000000
33 1000
34
```

Figure 4: Final Values of registers x0 - x31 after executing the program.

8 Key Design Decisions

- Combinational memory read for load correctness.
- Structural ALU implementation.
- Big-endian consistency.
- Clear separation of sequential and combinational logic.

9 Challenges Faced

The implementation process involved several significant technical challenges:

- Correctly assembling B-type immediates from non-contiguous instruction bits.
- Debugging negative branch offsets and ensuring sign-extension correctness.
- Identifying and fixing load timing hazards.

- Maintaining correct big-endian byte mapping.
- Avoiding duplicate module compilation during include-based submission.
- Ensuring deterministic behavior under single-cycle timing constraints.
- Coordinating multi-module integration without introducing control-signal mismatches.

10 Conclusion

Through modular development, staged testing, and systematic debugging, we implemented a complete and functionally correct 64-bit single-cycle RV64I subset processor. The final design correctly executes arithmetic, logic, memory, branch, and loop instructions and complies fully with the required submission format.